Министерство образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

# ОТЧЕТ

по лабораторной работе №7

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнили:

студенты группы 25ВВВ3

Шибанов Ф.Д.

Флеонов М.В.

Пенза 2026

**Цель работы:** Изучить алгоритм обхода графа в глубину, реализовать его рекурсивную версию для графа, представленного матрицей смежности и списками смежности, а также нерекурсивную версию с использованием стека.

**Описание алгоритмов**

Обход графа в глубину.

Обход в глубину (DFS, depth-first search) — способ последовательного посещения всех вершин графа, при котором движение продолжается от текущей вершины к ещё не посещённой смежной вершине, пока это возможно. Если у текущей вершины не осталось непосещённых соседей, выполняется возврат к вершине, из которой в неё пришли, и поиск продолжается оттуда. Для учёта посещённых вершин используется массив меток visited: перед обходом все метки сбрасываются в 0, при посещении вершины метка устанавливается в 1, благодаря чему каждая вершина выводится ровно один раз.

Если граф несвязный, обход из одной вершины посещает только её компоненту связности. Поэтому основная программа перебирает все вершины по порядку и запускает обход из каждой вершины, которая ещё не помечена. Так обходится весь граф, а число запусков равно числу компонент связности.

Генерация матрицы смежности.

Матрица смежности неориентированного графа симметрична, а её главная диагональ нулевая. Поэтому случайные значения 0 или 1 генерируются функцией rand() только для элементов над главной диагональю и сразу записываются в симметричную ячейку. Генератор инициализируется текущим временем, так что при каждом запуске получается новый граф.

Рекурсивный обход по матрице смежности.

Функция обхода получает номер вершины v, помечает её как посещённую и выводит. Затем просматривается строка v матрицы смежности: для каждой вершины i, которая смежна с v и ещё не посещена, выполняется рекурсивный вызов. Возврат по графу происходит автоматически при завершении вызова. Каждая вершина обрабатывается один раз, и для неё просматривается вся строка матрицы из n элементов, поэтому время обхода пропорционально $n^2$.

Списки смежности.

Список смежности вершины — односвязный список номеров её соседей; граф описывается массивом из n указателей на начала списков. Списки строятся по матрице смежности: строка просматривается слева направо, и для каждой

единицы в конец списка добавляется новый узел. Добавление в конец выполняется через указатель на поле next последнего узла, поэтому соседи в списке идут в порядке возрастания номеров. Рекурсивный обход по спискам отличается от обхода по матрице только способом перебора соседей: вместо просмотра всей строки проходится список. Поскольку суммарная длина всех списков равна удвоенному числу рёбер m, время обхода пропорционально $n + m$, что для разреженных графов существенно меньше $n^2$.

Нерекурсивный обход с использованием стека.

Рекурсия заменяется явным стеком, в котором хранится путь от начальной вершины до текущей. На каждом шаге берётся вершина с вершины стека и в её строке матрицы ищется первый непосещённый сосед. Если сосед найден, он помечается, выводится и помещается в стек — это соответствует рекурсивному вызову. Если соседей не осталось, вершина снимается со стека — это соответствует возврату из вызова. Чтобы не просматривать строку каждый раз с начала, в массиве pos для каждой вершины хранится позиция, с которой продолжается поиск соседей. Этот массив играет роль счётчика цикла, сохраняемого в кадре рекурсивного вызова, поэтому порядок обхода полностью совпадает с рекурсивной версией. Каждая вершина помещается в стек один раз, так что достаточно стека размером n.

**Ход работы**

**Задание 1**

Задание: сгенерировать с помощью генератора случайных чисел матрицу смежности неориентированного графа G и вывести её на экран; для сгенерированного графа выполнить процедуру обхода в глубину.

Число вершин вводится с клавиатуры в диапазоне от 1 до 20 с проверкой корректности ввода. Матрица смежности генерируется и выводится на экран. Обход реализован рекурсивной функцией dfs:

```
void dfs(int n, int v) {
    visited[v] = 1;
    printf("v%d ", v + 1);
    for (int i = 0; i < n; i++)
        if (adj[v][i] == 1 && !visited[i]) dfs(n, i);
}
```

Перед обходом массив меток сбрасывается функцией reset_visited, после чего в цикле по всем вершинам обход запускается из каждой непосещённой вершины.

**Задание 1.3***

Задание: реализовать процедуру обхода в глубину для графа, представленного списками смежности.

Узел списка описан структурой из номера вершины и указателя на следующий узел. Функция build_lists строит списки по матрице смежности, функция print_lists выводит их на экран в виде цепочек, заканчивающихся NULL. Обход выполняет функция dfs_list:

```
void dfs_list(int v) {
    visited[v] = 1;
    printf("v%d ", v + 1);
    for (Node *p = list[v]; p != NULL; p = p->next)
        if (!visited[p->v]) dfs_list(p->v);
}
```

В конце программы память, выделенная под узлы списков, освобождается функцией free_lists.

**Задание 2***

Задание: для матричной формы представления графа выполнить преобразование рекурсивной реализации обхода к нерекурсивной.

Нерекурсивный обход реализован функцией dfs_stack. Стек хранится в массиве stack, индекс top указывает на первую свободную ячейку. Основной цикл работает, пока стек не пуст:

```
while (top > 0) {
    int v = stack[top - 1];
    int i = pos[v];
    while (i < n && !(adj[v][i] == 1 && !visited[i])) i++;
    if (i < n) {
        pos[v] = i + 1;
        visited[i] = 1;
        printf("v%d ", i + 1);
        stack[top++] = i;
    } else {
        top--;
    }
}
```

Вершина v на вершине стека не снимается сразу, а только читается: поиск её следующего соседа продолжается с позиции pos[v]. Найденный сосед помещается в стек и становится текущей вершиной; если соседей больше нет, вершина v снимается со стека, и обход возвращается к предыдущей вершине пути.

Результат работы программы для графа из пяти вершин показан на рисунке 1, а сам граф с порядком посещения вершин — на рисунке 2.

```
[fedor@olegator lab7]$ ./lab7
Enter number of vertices (1..20): 5

=== Task 1 ===
Adjacency matrix:
    v1  v2  v3  v4  v5
v1  0   1   0   0   0
v2  1   0   1   0   0
v3  0   1   0   1   0
v4  0   0   1   0   1
v5  0   0   0   1   0
DFS order: v1 v2 v3 v4 v5

=== Task 1.3* ===
Adjacency lists:
v1 : v2 -> NULL
v2 : v1 -> v3 -> NULL
v3 : v2 -> v4 -> NULL
v4 : v3 -> v5 -> NULL
v5 : v4 -> NULL
DFS order: v1 v2 v3 v4 v5

=== Task 2* ===
DFS order: v1 v2 v3 v4 v5
```

Рисунок 1 — Результат работы программы для n = 5

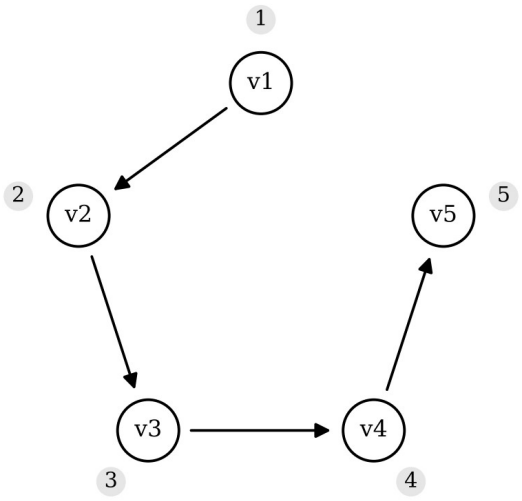



Рисунок 2 — Граф и порядок обхода в глубину

Сгенерированный граф представляет собой цепь v1 – v2 – v3 – v4 – v5. Обход начинается с v1, её единственный сосед v2 не посещён, поэтому происходит переход к нему; далее аналогично посещаются v3, v4 и v5. У v5 непосещённых соседей нет, после чего выполняется последовательный возврат до v1, и обход завершается. Все три реализации дали одинаковый порядок v1 v2 v3 v4 v5.

Совпадение порядков не случайно: соседи в списках смежности расположены по возрастанию номеров, то есть в том же порядке, в каком они встречаются в строке матрицы, а нерекурсивная версия за счёт массива pos воспроизводит порядок рекурсивных вызовов. Для дополнительной проверки программа была запущена на 2000 случайных графах с числом вершин от 1 до 20, и во всех случаях три реализации выдали одинаковый порядок обхода.

Характеристики реализаций сведены в таблицу 1.

Таблица 1 — Сравнение реализаций обхода в глубину

| **Реализация** | **Представление графа** | **Перебор соседей** | **Время работы** |
|---|---|---|---|
| dfs | матрица смежности | просмотр строки матрицы | $O(n^2)$ |
| dfs_list | списки смежности | проход по списку | $O(n + m)$ |
| dfs_stack | матрица смежности | просмотр строки с позиции pos[v] | $O(n^2)$ |

Рекурсивные версии короче и нагляднее, но используют системный стек вызовов, глубина которого при обходе длинной цепи равна числу вершин. Нерекурсивная версия хранит путь в собственном массиве, размер которого заранее известен и ограничен числом вершин, поэтому она не зависит от ограничений системного стека.

После выполнения заданий был скомпилирован исполняемый файл. Изменения закоммичены и загружены на GitHub.

Ссылка на репозиторий: https://github.com/TrueVibeCoder/LiOAvIZ-lab7

**Вывод:** В ходе лабораторной работы изучен алгоритм обхода графа в глубину. Реализована программа, которая генерирует случайную матрицу смежности неориентированного графа и выполняет обход тремя способами: рекурсивно по матрице смежности, рекурсивно по спискам смежности и нерекурсивно по матрице смежности с использованием стека. Для обхода несвязных графов

обход запускается из каждой непосещённой вершины. Установлено, что все три реализации дают одинаковый порядок посещения вершин; обход по матрице выполняется за время $O(n^2)$, а по спискам смежности — за $O(n + m)$, что выгоднее для разреженных графов. Замена рекурсии явным стеком с сохранением позиции поиска соседей позволяет точно воспроизвести порядок рекурсивного обхода.